INVENTORY MANAGEMENT

MR.PIBHU CHITBURANACHART
MR.PANNAWISH KRIENGYAKUL
MR.AKKHRAWIN NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

INVENTORY MANAGEMENT

MR.PIBHU CHITBURANACHART
MR.PANNAWISH KRIENGYAKUL
MR.AKKHRAWIN NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

Computer Engineering Project
entitled

INVENTORY MANAGEMENT

Mr.Pibhu Chitburanachart
Researcher

Mr.Pannawish Kriengyakul
Researcher

Mr.Akkhrawin Nair
Researcher

Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science),
Ph.D. (Computer Engineering)
Advisor

Thesis
entitled
INVENTORY MANAGEMENT

was submitted to the Faculty of Engineering & International College,
Mahidiol University
for the degree of Bachelor of Engineering (Computer Engineering)
on
July 15, 2024

_____
Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science),
Ph.D. (Computer Engineering)
Chair Committee

_____
Committee 1, Ph.D. (Computer Engineering)
Committee

_____
Committee 2, Ph.D. (Computer Engineering)
Comittee

BIOGRAPHY

NAME	Mr.Pibhu Chitburanachart
DATE OF BIRTH	29 August 2004
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	098 494 7984
E-MAIL	Pibhu22@gmail.com

NAME	Mr.Pannawish Kriengyakul
DATE OF BIRTH	17 April 2003
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	0989705604
E-MAIL	peto03.tcs@gmail.com

BIOGRAPHY

NAME	Mr.Akkhrawin Nair
DATE OF BIRTH	22 September 2002
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	0987819644
E-MAIL	akkhrawin22@gmail.com

ACKNOWLEDGEMENT

First and foremost, we would like to express our special thanks and sincere of gratitude to our advisor and co-advisors, Asst. Prof. Mingmanas Sivaraksa, Asst. Prof. Lalita Narupiyakul, Asst. Prof. Tanasanee Phienthrakul, who insight, advise and knowledge us a lot in finalizing this project within the limited time frame.

Beside our advisors, we would like to thank to the committees, Asst. Prof. XXX XXXX, Asst. Prof. YYY YYYY, and Asst. Prof. ZZZ ZZZZ, for their insightful comment and encouragement. Also, the questions which encourage us to rethink and research more about some factors which we missed. Therefore, this project would not be completed without comments, questions, and support from our committees.

Finally, we would like to special thanks to our university, Mahidol University International College, and Faculty of Engineering, Mahidol University, which provided us a lot of resources to study, financial means for researching this project.

Mr.Pibhu Chitburanachart
Mr.Pannawish Kriengyakul
Mr.Akkhrawin Nair

ชื่อโครงงาน	ชื่อวิทยานิพนธ์
ผู้วิจัย	นายพิภู จิตบูรณะชาติ ICCI
	นาย ปัณณวิชญ์ เกรียงยะกุล ICCI
	นายอัครวินทร์ แนร์ ICCI
	ปริญญา วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
อาจารย์ที่ปรึกษา	ผศ.□□. มิ่งมานัส ศิวรักษ์
สำเร็จการศึกษา	15 กรกฎาคม 2567

บทคัดย่อ

บทคัดย่อภาษาไทย (not required for your project proposal, but it is mandatory for the black book)

คำสำคัญ : ลาเท็ก / วิทยานิพนธ์ (~5 คำ)

27 หน้า

INVENTORY MANAGEMENT

STUDENTS	Mr. Pibhu Chitburanachart ICCI
	Mr. Pannawish Kriengyakul ICCI
	Mr. Akkhrawin Nair ICCI
DEGREE	Bachelor of Engineering (Computer Engineering)
PROJECT ADVISOR	Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science), Ph.D. (Computer Engineering)
DATE OF GRADUATION	July 15, 2024

ABSTRACT

This project presents the design and implementation of an Inventory Management System for small and medium-sized trading businesses. The system is intended for middle-man business workflows in which a company purchases products from suppliers, manages received stock, sells products to customers, and monitors operational documents related to receivables and payables.

The system was developed as a three-tier web application using React and Vite for the frontend, Django and Django REST Framework for the backend, and MySQL for relational data storage. The implementation includes master data management for products, categories, suppliers, and customers; purchase and sales transaction management; quotation records; billing notes; payment batches; credit notes; stock validation; dashboard summaries; and a read-only text assistant for selected operational questions.

The backend is responsible for authoritative business logic. Available stock is derived from received purchase line items minus committed sales line items rather than being manually stored as a static product value. Sales are validated server-side before they can move into stock-deducting statuses, and stock allocation is performed through received purchase layers using FIFO logic. Finance workflows are also controlled by backend eligibility endpoints so that billing notes, payment batches, and credit notes can only be created from valid transactions.

The result is a functional web-based inventory management system that integrates purchasing, sales, stock visibility, and financial document tracking in one application. The assistant feature is currently limited to stock and fulfillment questions, customer or supplier summaries, receivable and payable exception checks, and document reference or line-item lookup. The system supports daily SME trading operations and provides a foundation for future improvements such as expanded inventory planning formulas, role-based permissions, formal audit logs, and larger-scale production testing.

KEYWORDS : Inventory Management / SME Trading / Django REST Framework / React / MySQL / AI Assistant

27 Pages

CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

1.1 Background

Small and medium-sized trading businesses often operate as intermediaries between suppliers and customers. Their daily operations depend on purchasing goods from suppliers, receiving stock, selling goods to customers, preparing business documents, and monitoring the margin between purchase cost and sales revenue. In such businesses, inventory management is not only a matter of counting product quantities. It also requires the coordination of quotations, purchase orders, sales records, fulfillment status, receivables, payables, and stock availability.

Many small businesses still depend on spreadsheets, paper records, manual document folders, or general-purpose accounting tools. These methods may be workable for a small number of products, but they become difficult to control when the business must track multiple suppliers, product units, transaction statuses, cancelled lines, delayed purchases, customer billing, and supplier payments. Manual workflows also make it difficult to answer operational questions quickly, such as which products are low in stock, which sales can be billed, which purchases are ready for payment, or whether a sale can be fulfilled from available received stock.

The Inventory Management System was developed to address these practical workflow problems for an SME trading environment. The implemented system uses a React frontend, a Django REST Framework backend, and a MySQL relational database. Its design focuses on master data management, purchasing, sales, quotations, billing notes, payment batches, credit notes, dashboard summaries, stock validation, and read-only text queries for selected operational information. The backend is responsible for authoritative validation and stock calculations, while the frontend provides a compact web interface for daily operational work.

The project also includes an AI-assisted inventory chat feature with a limited scope. It is a read-only helper that can summarize supported backend data; it does not cre-

ate, edit, approve, cancel, or override transactions. Its current supported areas are stock and fulfillment questions, customer or supplier summaries, receivables and payables with exception checks, and reference or line-item lookup.

1.2 Objective

1. To develop a system that records purchase and sales transactions, including the current status of each process, with an attachment file function that links all supporting documents directly to the corresponding transactions.
2. To develop an inventory management system that displays the overall product stock in the warehouse and provides tools for calculating essential inventory information.
3. To integrate an AI chatbot using the ChatGPT API that connects to the system's backend data, allowing users to request supported operational summaries through text queries.

1.3 Scope

The scope of this project is limited to the implemented Inventory Management System codebase. The system covers the following functional areas:

1. **Master data management:** The system manages categories, products, suppliers, and customers. Products support SKUs, previous SKUs, product names, category links, unit conversions, reorder levels, active or disabled status, product pictures, and supplier-specific sourcing information.
2. **Purchasing workflow:** The system records purchase transactions, supplier information, purchase line items, expected delivery dates, received dates, item statuses, VAT totals, bill discounts, supplier tax invoice references, uploaded documents, and payable totals. Purchase status and purchase item status are synchronized by backend logic.
3. **Sales workflow:** The system records sales transactions, customer information, customer purchase-order references, line items, delivery statuses, uploaded documents, VAT totals, discounts, billing-note links, credit-note links, and source

quotation links. Stock is validated on the server before a sale can move into a stock-deducting status.

4. **Inventory and stock control:** Available stock is derived from received purchase quantities minus committed sale quantities. Sale allocations use received purchase layers and support first-in, first-out allocation logic. The inventory workspace and dashboard show stock position, reorder information, demand, purchase pipeline, stock value, and product history.
5. **Quotation workflow:** The system stores quotations with customer and supplier references, quotation validity information, shipping date, payment term, VAT mode, total amounts, normalized line items, and supplier options for quotation items. Quotations can be linked to derived purchases and sales.
6. **Finance workflow:** The system supports billing notes for customer receivables, payment batches for supplier payables, and credit notes for cancelled or returned sales lines. Eligibility endpoints prevent users from selecting transactions that should not be included in these finance documents.
7. **Dashboard and reporting:** The backend provides dashboard summary and segment endpoints for operational summaries such as finance, trends, product movement, cashflow, and order coverage.
8. **Authentication and access flow:** The backend includes JWT login, refresh, and authenticated user profile endpoints. The frontend supports login, token refresh, logout, and guest mode. Backend-wide authentication enforcement is controlled by the INVENTORY_REQUIRE_AUTH setting.
9. **AI-assisted operation:** The system includes a text-based inventory assistant endpoint. Its current scope is read-only support for stock and fulfillment questions, customer or supplier summaries, receivables and payables with exception checks, and reference or line-item lookup.

The following items are outside the confirmed implemented scope and should not be claimed as completed unless additional implementation or evidence is added:

- Full production deployment performance testing and high-volume concurrent-user testing.
- Voice-based or image-based chatbot interaction.

- Dedicated audit-log tables for every document status change.
- Confirmed implementation of Economic Order Quantity, Z-score, or a complete P-system calculation module.

1.4 Expected Results

The expected result of this project is a functional web-based inventory management system that can record purchase and sales transactions, track the status of each process, and link supporting documents directly to their corresponding transactions. The system is expected to provide an up-to-date view of stock levels based on received purchase quantities and committed sale quantities, together with clear traceability between products, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes.

The system is also expected to improve operational accessibility by providing dashboard summaries and a limited AI-assisted text interface. Users should be able to retrieve supported information such as product stock, selected transaction references, receivables, payables, partner summaries, and stock exceptions without manually searching every module. Overall, the completed system is expected to reduce manual checking, improve stock visibility, and support better day-to-day decision making for an SME trading business.

1.5 Timeline

Table 1.1 Project Timeline

Plan	Timeline								
	2024				2025				
	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May
1: Initiation									
2: Data Analyze									
3: Requirements									
4: Data Setup									
5: Development									
6: Integration									
7: User Test									
8: Final Draft									
9: Submittion									

CHAPTER 2
METHODOLOGY

This chapter explains how the Inventory Management System was designed and developed. The methodology is based on the implemented source code, including the React frontend, Django REST Framework backend, MySQL database configuration, REST API communication layer, business services, and backend test suite.

2.1 System Development Approach

The system was developed as a full-stack web application for SME trading businesses. The development approach separated the project into three main layers: a client-side user interface, a backend API and business logic layer, and a relational database layer. This structure allowed the frontend to focus on user interaction while the backend remained responsible for data validation, stock calculation, transaction eligibility, and financial workflow enforcement.

The project followed an incremental development approach. Master data modules were implemented first because products, categories, suppliers, and customers are required by later transaction workflows. After that, purchase and sales workflows were added, followed by quotations, billing notes, payment batches, credit notes, dashboard summaries, and the read-only inventory assistant.

2.2 Requirement Analysis

The requirement analysis was based on the daily operation of a middle-man SME trading business. The main requirement was to support the relationship between incoming stock from suppliers and outgoing sales to customers. The system therefore needed to maintain master records, record transaction documents, validate stock availability, track status changes, and summarize receivables and payables.

Table 2.1 Requirement Summary

Requirement Area	Implemented Response
Master data	Categories, products, suppliers, and customers are implemented as reusable records.
Inventory visibility	Product stock is calculated from received purchase items and committed sale items.
Purchasing	Purchase orders store supplier details, line items, receiving status, documents, VAT totals, discounts, and payable amount.
Sales	Sales records store customer details, line items, delivery status, documents, VAT totals, discounts, and links to billing or credit notes.
Quotations	Quotations store normalized line items, validity settings, supplier options, and links to derived purchase or sales records.
Receivables and payables	Billing notes track customer receivables, while payment batches track supplier payables.
Returns and cancellations	Credit notes are created from eligible cancelled or returned sale lines.
AI support	A text-based assistant answers supported read-only questions from backend data. Its current scope is limited to stock and fulfillment, partner summaries, receivable and payable exceptions, and reference or line-item lookup.

2.3 System Architecture

The system uses a three-tier web architecture. The frontend runs as a React single-page application. It communicates with the backend through REST API calls. The backend is implemented with Django and Django REST Framework and stores data in MySQL.

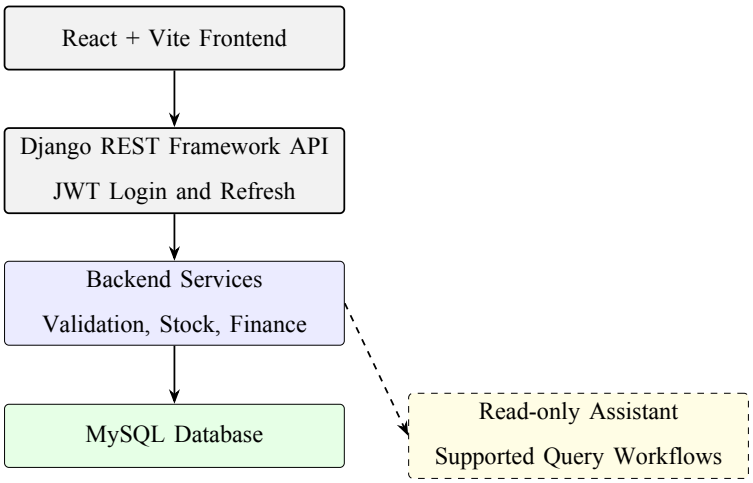

Figure 2.1 System architecture overview

Table 2.2 Architecture Components

Layer	Implementation	Responsibility
Presentation layer	React and Vite	Provides tabbed user interfaces for dashboard, inventory, transactions, finance documents, master data, and AI chat.
API layer	Django REST Framework	Exposes REST endpoints, serializers, filters, pagination, eligibility checks, and validation errors.
Business logic layer	Django service functions	Calculates stock, FIFO allocation, dashboard metrics, payable totals, finance summaries, and assistant context for supported read-only questions.
Data layer	MySQL through Django ORM	Stores normalized master data, transactions, line items, document links, finance records, and historical snapshot fields.
Authentication layer	Simple JWT	Supports login, access token refresh, and authenticated user profile retrieval.

2.4 Frontend Methodology

The frontend was implemented with React components, hooks, and helper modules. The application shell is organized around tabs that load the main pages: dashboard, inventory, AI chat, purchases, payment batches, quotations, sales, billing notes, credit notes, products, categories, suppliers, and customers.

API communication is centralized in the frontend API client. This client builds query strings, sends JSON or form-data payloads, attaches bearer tokens when available, refreshes expired access tokens, and dispatches an authentication-expired event when refresh fails. Data loading is organized through hooks that request dashboard data, lookup data, paginated lists, and eligibility results.

The frontend also maintains mock-data fallback behavior. If the backend is unavailable, parts of the application can still show mock data for development or demonstration. This fallback behavior is intentionally separate from backend-authoritative validation.

The user interface uses compact forms, directory tables, status controls, detail modals, and document reference components. User-facing labels are rendered through the language helper and translation dictionaries to support English and Thai text.

2.5 Backend Methodology

The backend was implemented as a Django project with one main inventory application. Django REST Framework viewsets expose CRUD endpoints for the primary resources, while function-based API views provide dashboard summaries, lookup lists, eligibility lists, product history, stock layers, and chat responses.

Serializers transform request payloads into model records and apply domain validation. For purchases and sales, serializers normalize line items, calculate base quantities, handle file uploads, update documents, and delegate stock or payable synchronization to backend service functions. Viewsets apply common search, status, date, party, product, amount, and VAT filters.

Backend services contain the main business calculations. These include stock metric snapshots, available stock layers, FIFO allocation, sale stock validation, purchase

payable total recalculation, payment batch total synchronization, dashboard segment generation, and context creation for the supported assistant workflows.

2.6 Database Design Methodology

The database was designed as a relational schema managed by Django migrations. Core master data is separated from transaction data. Categories, products, suppliers, and customers are maintained as master records, while purchases, sales, quotations, billing notes, payment batches, and credit notes reference them through foreign keys where appropriate.

The schema also preserves historical snapshot fields. For example, purchase and sale line items keep product names, SKUs, prices, quantities, and totals. Purchase and sale headers keep supplier or customer names. These fields allow historical business documents to remain readable even if a master record is later edited or disabled.

Table 2.3 Primary Database Model Groups

Group	Models
Master data	Category, Supplier, Customer, Product, ProductPicture, ProductUnitConversion, ProductSupplier
Purchasing	Purchase, PurchaseItem, PurchaseDocument
Sales	Sale, SaleItem, SaleItemAllocation, SaleDocument
Quotation	Quotation, QuotationItem, QuotationItemSupplier
Finance	BillingNote, BillingNoteLine, PaymentBatch, PaymentBatchLine, CreditNote, CreditNoteLine

2.7 API Design

The API follows REST principles for primary resources and specialized endpoints for operational summaries. CRUD resources are exposed through Django REST Framework routers. Additional endpoints are used where a simple resource endpoint is not sufficient, such as dashboard summaries, lookup lists, stock layers, product transaction history, eligibility checks, and chat.

Pagination is opt-in. If a list endpoint is called without a page parameter, it can

return a plain array for compatibility with existing frontend code. If a page parameter is provided, the response includes count, next, previous, page, page_size, total_pages, and results.

Table 2.4 Selected API Endpoint Groups

Endpoint Group	Purpose
/api/products/, /api/categories/	Product and category master data management.
/api/suppliers/, /api/customers/	Business partner master data management.
/api/purchases/, /api/sales/	Purchase and sales transaction workflows.
/api/quotations/	Quotation records and normalized quotation items.
/api/billing-notes/, /api/payment-batches/, /api/credit-notes/	Receivable, payable, and credit-note workflows.
/api/eligibility/...	Server-side lists of transactions eligible for billing notes, payment batches, or credit notes.
/api/dashboard/, /api/dashboard/segment/	Dashboard summaries and segmented reporting.
/api/chat/	Text-based read-only assistant for supported inventory, finance, partner-summary, and reference-lookup questions.

2.8 Business Workflow Design

The main workflow begins with master data. Products are categorized and connected to suppliers and customers through transaction records. Purchases create incoming stock layers when line items are received. Sales consume stock only when line items reach packed, shipped, or delivered statuses. This prevents draft or pending orders from reducing available stock.

The quotation workflow allows users to prepare commercial proposals before creating purchase or sales records. Billing notes are created only from eligible shipped

or delivered sales that are not already attached to an active billing note. Payment batches are created only from eligible received or partially received purchases that are not already attached to an active payment batch. Credit notes are created from cancelled or returned sale items that have not already been credited.

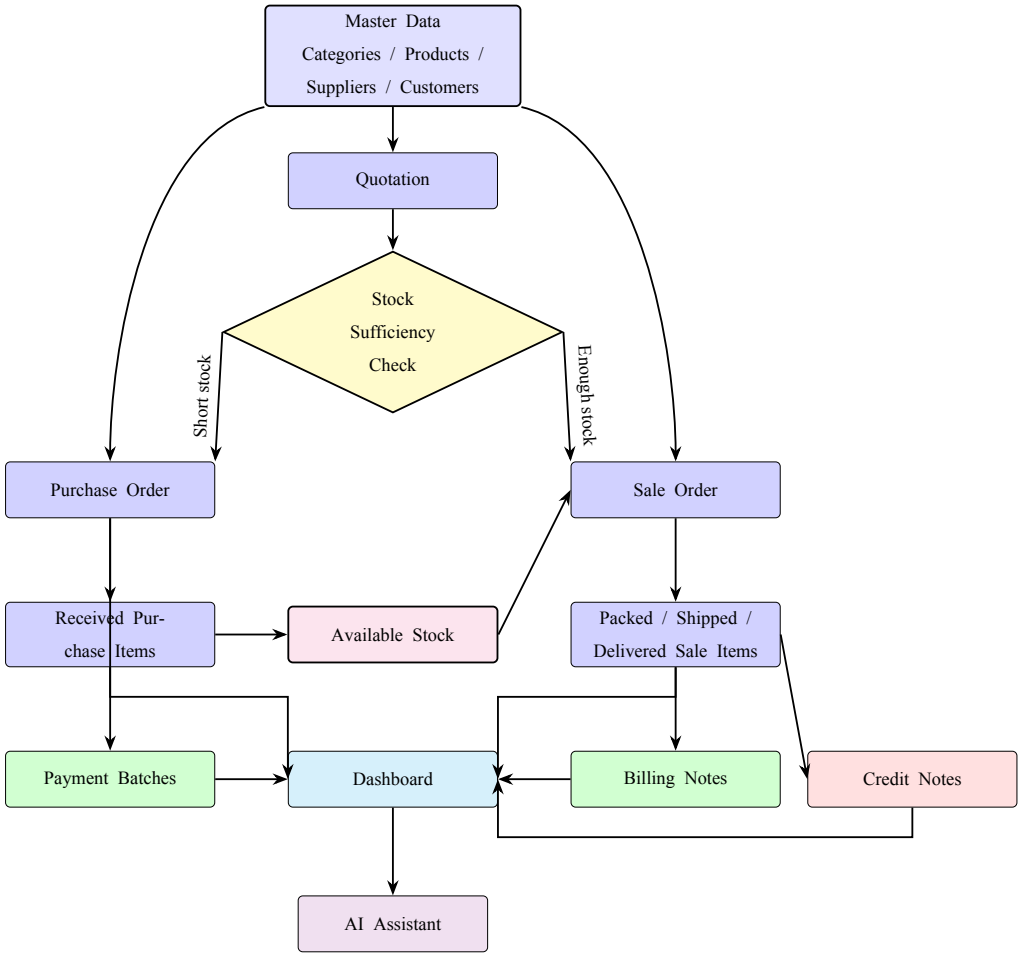

Figure 2.2 Business workflow design

2.9 Validation and Security Design

Validation is enforced primarily on the backend. The frontend provides usability checks, but the backend is the authoritative layer for stock validation, active product validation, percentage discount limits, document eligibility, purchase payable synchronization, sale allocation validity, and delete restrictions.

Security is implemented with JSON Web Tokens through Simple JWT. The backend exposes login and refresh endpoints. The frontend stores tokens in local storage or session storage depending on the login option and automatically refreshes access

tokens when needed. The authenticated profile endpoint requires authentication. Full API-wide authentication can be enabled through the INVENTORY_REQUIRE_AUTH environment setting.

The backend also includes production-oriented safety settings. It requires a non-default secret key when debug mode is disabled, restricts CORS origins through configuration, and uses Django's ORM and serializers instead of ad hoc SQL construction.

2.10 Testing Methodology

The backend test suite validates important business behavior using Django and Django REST Framework test cases. The tests cover pagination, product picture upload, operational data clearing, sale stock validation, sale item allocation, purchase item status behavior, relational quotation storage, reference number generation, eligibility endpoints, credit notes, operational seed data, quotation supplier options, payable synchronization, and assistant response alignment.

Table 2.5 Testing Summary

Test Area	Purpose
Pagination tests	Confirm that unpaginated and paginated list responses behave correctly.
Stock validation tests	Confirm that sales cannot commit more stock than available received layers.
Allocation tests	Confirm FIFO and manual stock allocation behavior.
Status tests	Confirm purchase and sale document statuses follow line item status rules.
Eligibility tests	Confirm billing note, payment batch, and credit note selection rules.
Financial tests	Confirm payable totals and linked payment batch amounts stay synchronized.
Assistant tests	Confirm that supported chat responses align with backend operational data.

2.11 Tools and Technologies Used

Table 2.6 Tools and Technologies

Category	Tool or Technology	Use in Project
Frontend framework	React 18	Builds the single-page application interface.
Frontend build tool	Vite	Provides local development and production build tooling.
Backend framework	Django 5	Provides the web backend, ORM, settings, migrations, and application structure.
API framework	Django REST Framework	Provides serializers, viewsets, API views, parsing, responses, and tests.
Authentication	Simple JWT	Provides access and refresh token authentication.
Database	MySQL	Stores relational business records and transaction history.
Assistant integration	Backend chat service	Builds compact backend context for supported read-only assistant responses.
Styling	CSS modules by domain file	Provides compact inventory application styling.
Testing	Django TestCase and DRF APITestCase	Validates backend business behavior and API behavior.

CHAPTER 3
RESULTS

This chapter reports the results of the implemented Inventory Management System. The results are based on the current source code and implemented workflows. Screenshots are represented as placeholders so that final approved application screenshots can be inserted later.

3.1 Main System Features

The completed system provides a web-based inventory management application for SME trading operations. The implemented functionality covers master data, inventory control, purchasing, sales, quotations, finance documents, dashboard reporting, and limited read-only assistant text queries.

Table 3.1 Implemented System Modules

Module	Implemented Result	Status
Dashboard	Shows operational summaries, segmented dashboard data, stock health, finance, trends, cashflow, product movement, and order coverage from backend services.	Implemented
Inventory	Shows stock position, reorder information, product stock rows, references, and inventory detail views.	Implemented
Products	Supports product CRUD, SKUs, previous SKUs, categories, unit conversions, reorder levels, active status, product pictures, and product history loading.	Implemented
Categories	Supports nested category management and full category tree viewing.	Implemented
Suppliers	Supports supplier master records, contact information, procurement contacts, branches, shipping addresses, and payment terms.	Implemented
Customers	Supports customer master records, contact information, branches, shipping addresses, and billing/payment term information.	Implemented
Purchases	Supports purchase records, line items, receiving statuses, payable totals, VAT totals, discounts, documents, product supplier syncing, and payment batch links.	Implemented
Sales	Supports sales records, line items, delivery statuses, stock validation, FIFO or manual allocation, VAT totals, discounts, documents, billing-note links, and credit-note links.	Implemented
Quotations	Supports quotations, normalized line items, supplier options, validity dates, payment terms, shipping dates, and derived purchase or sale links.	Implemented
Billing Notes	Supports receivable grouping from eligible	Implemented

3.2 Screenshot Placeholders

The following figures show the planned screenshot positions for the final report. The placeholders should be replaced with final application screenshots after the interface and sample data are approved.

TODO: Insert screenshot of inventory dashboard here.

Figure 3.1 Inventory dashboard

TODO: Insert screenshot of inventory control page showing stock rows and reorder information.

Figure 3.2 Inventory control page

TODO: Insert screenshot of product form with unit conversions and product picture support.

Figure 3.3 Product management form

TODO: Insert screenshot of purchase form or purchase history showing receiving statuses.

Figure 3.4 Purchase workflow

TODO: Insert screenshot of sales form showing delivery statuses and stock allocation.

Figure 3.5 Sales workflow

TODO: Insert screenshot of quotation entry page.

Figure 3.6 Quotation entry

TODO: Insert screenshot of billing note eligible sales or billing note detail.

Figure 3.7 Billing note workflow

TODO: Insert screenshot of payment batch eligible purchases or payment batch detail.

Figure 3.8 Payment batch workflow

TODO: Insert screenshot of a supported read-only assistant response, such as stock status or reference lookup.

Figure 3.9 Read-only inventory assistant

3.3 Feature Results

3.3.1 Master Data Results

The system implements master data screens and backend endpoints for categories, products, suppliers, and customers. Categories support parent-child relationships. Products support product display IDs, SKUs, previous SKUs, names, sub-names, unit settings, category references, reorder levels, active status, pictures, unit conversion rows, and product supplier links. Supplier and customer records store company names, locations, emails, telephone numbers, taxpayer IDs, branches, shipping addresses, remarks, and payment term information.

The implemented design separates master records from transaction records while preserving transaction snapshot fields. This allows historical purchases, sales, and quotations to remain readable even if a product, supplier, or customer is later edited.

3.3.2 Purchase Results

The purchase module implements purchase headers, purchase line items, receiving status, expected delivery date, received date, document upload, VAT mode, bill discount, totals, and payable total. The backend automatically recalculates payable totals so cancelled purchase lines are excluded from the amount still owed to the supplier.

Purchase status behavior is derived from purchase item statuses. Received purchase items increase available stock. Pending and cancelled purchase items do not increase stock. The backend also synchronizes product-supplier sourcing links from received or active purchase data.

3.3.3 Sales Results

The sales module implements customer sales transactions, line items, delivery status tracking, document upload, VAT mode, bill discount, totals, customer purchase-order reference, billing note links, credit note links, and source quotation links. Sales line items support pending, packed, shipped, delivered, cancelled, and returned statuses.

Server-side stock validation prevents a sale from committing more product quantity than available received stock. When a sale item reaches a stock-deducting status, the backend creates stock allocations from available purchase layers. If the user does not supply manual allocations, the backend uses FIFO allocation. If manual allocations are supplied, they must match the sale quantity exactly and must reference valid received stock layers.

3.3.4 Quotation Results

The quotation module implements quotation headers, validity dates, validity duration modes, customer and supplier links, shipping date, payment terms, VAT mode, total values, and line items. The database stores quotation lines in the normalized QuotationItem table, and supplier choices for quotation items are stored in QuotationItemSupplier. The API still exposes an items array to keep the frontend interface simple.

3.3.5 Finance Results

The finance workflow consists of billing notes, payment batches, and credit notes. Billing notes group eligible sales for customer receivable tracking. Payment batches group eligible purchases for supplier payable tracking. Credit notes reduce customer balances for sale items that are cancelled or returned.

The backend provides eligibility endpoints to ensure that only valid transactions can be selected. For example, active billing note lines prevent the same sale from being selected again for another active billing note. Active payment batch lines prevent the same purchase from being selected again for another active payment batch.

3.3.6 Dashboard and Assistant Results

The dashboard backend provides overall and segmented operational summaries. These include stock report rows, finance information, product movement, trends, cashflow, and order coverage. The assistant uses backend data for a limited set of read-only text queries: stock and fulfillment, customer or supplier summaries, receivable and payable exceptions, and reference or line-item lookup. It is not implemented as a general-purpose chatbot and does not create or modify records.

3.4 Implemented Features Table

Table 3.2 Implemented Feature Summary

Feature	Evidence from Implementation
REST API communication	Frontend API client calls backend endpoints for dashboard, lookups, products, purchases, sales, quotations, billing notes, payment batches, credit notes, auth, and chat.
JWT authentication	Backend exposes login, refresh, and authenticated profile endpoints; frontend stores and refreshes tokens.
Opt-in pagination	Backend pagination returns paginated shape only when page parameters are supplied.
Stock validation	Sale serializer calls backend stock issue detection before committing stock-deducting sale statuses.
FIFO allocation	Backend service creates sale item allocations from oldest available received purchase layers.
Manual allocation	Sale item payloads may include allocation requests, which are validated against stock layers.
Eligibility filtering	Backend eligibility endpoints return valid sales for billing notes, valid purchases for payment batches, and valid sale lines for credit notes.
Normalized quotations	Quotation API exposes items while database stores quotation lines and supplier options in relational tables.
Assistant	Chat endpoint builds compact backend context and returns text responses for supported read-only operational questions.

3.5 Key Findings from the Implementation

The implementation shows that a trading-focused inventory system requires tight coordination between purchases and sales. Stock cannot be treated as a manually edited product field because stock availability depends on received purchase items and committed sales. The backend service layer is therefore essential for accurate inventory behavior.

Another key finding is that eligibility endpoints simplify finance workflows. Billing notes and payment batches depend on transaction status and prior selection. Moving this logic to the backend reduces the risk of duplicate billing or duplicate supplier payment grouping.

The implementation also shows the importance of historical snapshots. Although products, suppliers, and customers are relational master records, transaction records keep names, SKUs, prices, and totals so that business documents remain readable after master data changes.

3.6 Discussion of Objective Achievement

The first objective was to record purchase and sales transactions with status tracking and document attachment support. This objective is met by the purchase and sales modules, which include transaction headers, line items, status fields, file upload handling, and document reference behavior.

The second objective was to provide inventory visibility. This objective is met through backend-derived stock calculations, product metrics, stock layers, product history, inventory pages, reorder levels, and dashboard stock summaries. However, formulas such as EOQ, Z-score, and complete P-system calculations were not identified in the current codebase and should be treated as future work unless implemented later.

The third objective was to integrate an AI chatbot connected to backend data. This objective is met within the current implemented scope by the chat endpoint and frontend chat panel. The assistant answers only supported read-only text questions from backend context: stock and fulfillment, partner summaries, receivable and payable exceptions, and reference or line-item lookup.

3.7 Limitations Found During Development

The implementation has several limitations that should be considered in the final evaluation:

- The codebase includes backend tests, but final executed test logs and measured test results must still be inserted into the report.

- The system supports JWT authentication, but backend-wide authentication is configurable and may be disabled in local development.
- The frontend includes guest and mock-data behavior for demonstration, so final evaluation should distinguish between mock data and backend-connected behavior.
- The codebase does not show confirmed implementation of EOQ, Z-score, or full P-system inventory formulas.
- High-volume concurrent-user testing and live production performance evaluation were not identified in the repository.
- A dedicated audit-log model for every document status change was not identified.
- Screenshot evidence still needs to be inserted and verified.

CHAPTER 4
CONCLUSION

4.1 Brief Project Overview

The Inventory Management System is a full-stack web application developed for SME trading businesses that purchase products from suppliers and resell them to customers. The system supports product and partner master data, purchase workflows, sales workflows, quotation preparation, inventory control, billing notes, payment batches, credit notes, dashboard reporting, and limited read-only assistant text queries.

The project was implemented using a React and Vite frontend, a Django REST Framework backend, and a MySQL relational database. The system emphasizes backend-authoritative validation so that stock, eligibility, and financial workflows remain consistent even when requests are made directly to the API.

4.2 Objective Summary

The project objectives focused on recording purchase and sales transactions, showing inventory stock information, and integrating an AI chatbot connected to backend data. The implemented system satisfies these objectives through transaction modules, stock calculation services, dashboard and inventory pages, and a text-based assistant limited to supported read-only operational summaries.

4.3 System Design Summary

The system was designed using a three-tier architecture. The React frontend provides a tabbed user interface for operational work. The Django REST Framework backend exposes REST APIs, serializers, filters, pagination, authentication endpoints, eligibility endpoints, and dashboard endpoints. The MySQL database stores normalized master data, transaction line items, finance documents, stock allocation records, and historical snapshot fields.

The design separates master records from transaction records. Products, categories, suppliers, and customers are managed independently, while purchases, sales, quotations, billing notes, payment batches, and credit notes reference those records and preserve snapshot values for audit readability.

4.4 Implementation Summary

The frontend implementation includes pages for dashboard, inventory, AI chat, purchase history, sales history, quotations, billing notes, payment batches, credit notes, products, categories, suppliers, customers, and settings. It also includes a centralized API client, authentication context, translation dictionaries, reusable directory filters, pagination controls, and mock-data fallback behavior.

The backend implementation includes Django models, migrations, serializers, viewsets, business services, pagination, authentication views, management commands, and tests. The most important backend services include stock metric calculation, FIFO allocation, sale stock validation, purchase payable recalculation, payment batch synchronization, dashboard segment generation, and context preparation for supported assistant questions.

4.5 Result Summary

The implemented result is a functional inventory management system that connects purchasing, sales, finance, and stock visibility. Purchases create available stock only when purchase items are received. Sales reduce stock only when sale items reach packed, shipped, or delivered statuses. The system validates stock availability before allowing sales to commit unavailable quantities.

The system also supports receivable and payable workflows. Billing notes are created from eligible sales, payment batches are created from eligible purchases, and credit notes are created from eligible cancelled or returned sale lines. The dashboard provides summary views, while the assistant provides read-only responses for a limited set of supported operational questions.

4.6 Key Findings

Several key findings were identified during implementation:

1. Stock should be derived from transaction status rather than manually stored as a product field.
2. Backend validation is necessary because frontend checks alone cannot protect direct API calls.
3. FIFO stock allocation provides a practical method for connecting sales cost to received purchase layers.
4. Eligibility endpoints reduce duplicate billing, duplicate payment grouping, and invalid credit-note creation.
5. Historical snapshot fields are necessary for preserving the readability of old business documents.
6. AI-assisted text queries are useful only when kept within their current supported scope and grounded in backend data.

4.7 Obstacles and Challenges

The main challenge was coordinating many related business workflows without allowing inconsistent data. Purchases, sales, billing notes, payment batches, and credit notes are not isolated modules. A status change in one workflow can affect stock, payable amounts, receivable eligibility, or credit-note eligibility.

Another challenge was balancing frontend usability with backend authority. The frontend needed to be simple enough for daily use while the backend still had to enforce strict validation. This required duplicated awareness of some concepts in the UI, but final authority remained in serializers and service functions.

The project also required careful database design. The schema needed normalized relationships for maintainability while preserving historical snapshot fields for business-document accuracy. Removing snapshot fields would simplify the schema but would weaken audit readability.

4.8 Future Work

Future work should focus on improving evaluation evidence, extending inventory planning calculations, and strengthening operational controls.

1. Add final approved screenshots and user testing results to the report.
2. Run and record final backend checks, migration checks, backend tests, frontend build checks, and security audit results.
3. Implement and validate EOQ, Z-score, and full P-system calculations if they remain part of the final project objective.
4. Add a dedicated audit-log model for document status changes and important financial workflow updates.
5. Add role-based permissions if the system is deployed for multiple staff roles.
6. Expand production performance testing with larger datasets and concurrent users.
7. Improve assistant evaluation by comparing supported answers against expected backend records.
8. Add more formal reporting exports for management, accounting, and inventory review.

REFERENCES

[1] .

[2] .

[3] .

[4] S. Axsäter. Inventory control. 2006.

[5] M. Muller. Essentials of inventory management. 2011.